# Algorithm Interview Reference — Quick Reference Guide

## Table of Contents


1. Union-Find
2. Binary Search
3. Two Pointers
4. Sliding Window
5. Monotonic Stack
6. Intervals
7. Top-K & Heaps
8. Greedy Algorithms
9. Dynamic Programming
10. Graph Algorithms
11. Tree Algorithms
12. Backtracking
13. String Algorithms
14. Java Standard Library


---

# 1. Union-Find 🔗

**When to use**: Connected components, cycle detection, dynamic connectivity
**Time**: O(α(n)) per operation with path compression
**Space**: O(n)

**Key Operations**: `find(x)`, `union(x, y)`, `connected(x, y)`

```
public class UnionFind {
    private int[] parent, rank;
    private int count;

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
```

```
        count = n;
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    public int find(int i) {
        if (parent[i] != i) parent[i] = find(parent[i]); // Path com
        return parent[i];
    }

    public void union(int i, int j) {
        int rootI = find(i), rootJ = find(j);
        if (rootI != rootJ) {
            if (rank[rootI] > rank[rootJ]) parent[rootJ] = rootI;
            else if (rank[rootI] < rank[rootJ]) parent[rootI] = root
            else { parent[rootJ] = rootI; rank[rootI]++; }
            count--;
        }
    }

    public boolean connected(int i, int j) { return find(i) == find(
    public int getCount() { return count; }
}
```

**Known Tricks**:

- Path compression in find()
- Union by rank for balanced trees
- Count connected components

---

# 2. Binary Search 🔍

**When to use**: Sorted arrays, search spaces, optimization problems
**Time**: O(log n)
**Space**: O(1)

```
// Standard Binary Search
public int binarySearch(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) left = mid + 1;
```

```
            else right = mid - 1;
        }
        return -1;
    }

    // Lower Bound (first position >= target)
    public int lowerBound(int[] nums, int target) {
        int left = 0, right = nums.length;
        while (left < right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] < target) left = mid + 1;
            else right = mid;
        }
        return left;
    }
```

**Known Tricks**:

- Avoid overflow: `mid = left + (right - left) / 2`
- Lower bound: `if (arr[mid] < target) left = mid + 1; else right = mid;`
- Upper bound: `if (arr[mid] <= target) left = mid + 1; else right = mid;`
- Search range: Use lower bound + upper bound

---

# 3. Two Pointers ↔️

**When to use**: Sorted arrays, pairs/triplets, cycle detection, array partitioning
**Time**: O(n)
**Space**: O(1)

**Patterns**:

- **Opposite ends**: `left = 0, right = n-1; move based on sum`
- **Same direction**: Fast/slow pointers for cycles
- **Partitioning**: Separate elements by condition

**Known Tricks**:

- Three Sum: Fix one element, use two pointers on rest
- Remove duplicates: Skip `while (nums[i] == nums[i+1]) i++`

- Dutch National Flag: Use 3 pointers for 3-way partition
- Cycle detection: Floyd's algorithm (tortoise and hare)

---

# 4. Sliding Window 📊

**When to use**: Subarray/substring problems, fixed/variable window constraints
**Time**: O(n)
**Space**: O(k) for tracking characters

**Patterns**:

- **Fixed window**: Calculate first window, then slide
- **Variable window**: Expand right, contract left when invalid
- **At most K**: Use map to track frequencies

**Known Tricks**:

- Contract window: `while (condition_violated) { remove_left_element; left++; }`
- Frequency map: `map.put(char, map.getOrDefault(char, 0) + 1)`
- Valid window: Track number of valid characters vs required

---

# 5. Monotonic Stack 📈

**When to use**: Next greater/smaller element, histogram problems, temperature problems
**Time**: O(n)
**Space**: O(n)

**Principle**: Maintain elements in monotonic order (increasing/decreasing)

**Known Tricks**:

- Store indices instead of values for distance calculations
- **Next greater**: Pop while `stack.peek() < current`
- **Previous smaller**: Same logic, iterate backwards
- **Histogram**: Add sentinel 0 at end to flush remaining elements

---

# 6. Intervals 🗓️

**When to use**: Meeting rooms, scheduling, overlapping ranges
**Time**: O(n log n) for sorting
**Space**: O(n)

**Standard approach**: Sort by start time, then merge/process

**Known Tricks**:

- **Merge**: `if (current[1] >= next[0]) merge else add current`
- **Insert**: Process non-overlapping before/after separately
- **Meeting rooms**: Use min heap to track end times
- **Activity selection**: Sort by end time, greedy select

---

# 7. Top-K & Heaps ⛰️

**When to use**: Kth largest/smallest, streaming data, priority processing
**Time**: O(n log k)
**Space**: O(k)

**Principle**: Use heap of size K to maintain top/bottom K elements

**Known Tricks**:

- **Kth largest**: Use min heap of size K
- **Kth smallest**: Use max heap of size K
- **Streaming median**: Use two heaps (max heap for smaller half, min heap for larger)
- **K closest points**: Use max heap with distance comparison

---

# 8. Greedy Algorithms 🎯

**When to use**: Activity selection, scheduling, optimization with greedy choice property

**Time**: Varies (often O(n log n) due to sorting)

**Principle**: Make locally optimal choice at each step

**Common patterns**:

- **Activity selection**: Sort by end time
- **Interval scheduling**: Greedy by earliest finish time
- **Jump game**: Always jump to furthest reachable position

**Known Tricks**:

- Prove greedy choice is safe
- Sort by the "limiting" constraint
- Consider exchange argument for optimality proof

---

# 9. Dynamic Programming 🔄

**When to use**: Optimization problems, counting, overlapping subproblems
**Time**: $O(n^2)$ typical
**Space**: O(n) or $O(n^2)$

**Steps**:

1. Define state
2. Find recurrence relation
3. Identify base cases
4. Determine computation order

**Patterns**:

- **1D DP**: `dp[i] = f(dp[i-1], dp[i-2], ...)`
- **2D DP**: `dp[i][j] = f(dp[i-1][j], dp[i][j-1], ...)`
- **LIS**: `dp[i] = max(dp[j] + 1)` where `arr[j] < arr[i]`

**Known Tricks**:

- Space optimization: Only keep previous row/column
- Bottom-up vs top-down approach
- Memoization for recursive solutions

# 10. Graph Algorithms

**Representations**:

- **Adjacency List**: `List<List<Integer>>` - O(V + E) space
- **Adjacency Matrix**: `int[][]` - O($V^2$) space

**Traversal**:

- **BFS**: Layer by layer using queue - shortest path in unweighted
- **DFS**: Go deep using recursion/stack - detect cycles, topological sort

**Shortest Path**:

- **Dijkstra**: Non-negative weights, O((V + E) log V)
- **Bellman-Ford**: Negative weights, O(VE)

**MST**:

- **Kruskal**: Sort edges + Union-Find, O(E log E)
- **Prim**: Start from vertex + priority queue, O((V + E) log V)

**Cycle Detection**:

- **Undirected**: DFS with parent tracking
- **Directed**: DFS with 3-coloring (WHITE/GRAY/BLACK)

**Known Tricks**:

- Use Set in adjacency list to avoid duplicate edges
- BFS for shortest unweighted path
- Topological sort: Kahn's algorithm or DFS post-order

# 11. Tree Algorithms

**Traversals**:

- **Inorder**: Left → Root → Right (sorted order for BST)
- **Preorder**: Root → Left → Right (copy structure)

- **Postorder**: Left → Right → Root (delete nodes)

**Properties**:

- **Height**: `max(left_height, right_height) + 1`
- **Diameter**: `left_height + right_height`
- **Balanced**: Height difference ≤ 1

**BST Operations**:

- **Validate**: Use min/max bounds
- **Insert**: Recursive placement
- **Delete**: Handle 3 cases (leaf, one child, two children)

**Known Tricks**:

- **LCA**: Return when you find either node, check if both subtrees return non-null
- **Path sum**: Use backtracking with path list
- **Serialize**: Use preorder with null markers

---

# 12. Backtracking

**When to use**: Generate all solutions, permutations, combinations, constraint satisfaction

**Template**:

```
if (goal_reached) { add_solution; return; }
for (choice in choices) {
    make_choice;
    backtrack;
    undo_choice;
}
```

**Patterns**:

- **Permutations**: Use boolean array to track used elements
- **Combinations**: Use start index to avoid duplicates
- **Subsets**: Include current element or not

**Known Tricks**:

- Sort input for duplicate handling
- Use start index to avoid repeated combinations
- Swap-based permutations for space efficiency
- Skip duplicates: `if (i > 0 && nums[i] == nums[i-1] && !used[i-1]) continue`

---

# 13. String Algorithms

**Pattern Matching**:

- **KMP**: O(n + m) with LPS preprocessing
- **Rabin-Karp**: O(n) average with rolling hash
- **Boyer-Moore**: O(n/m) best case

**Trie (Prefix Tree)**:

- **Insert/Search**: O(L) where L is string length
- **Autocomplete**: DFS from prefix node
- **Word break**: DP + Trie lookup

**Known Tricks**:

- **Rolling hash**: `hash = (hash * base + char) % mod`
- **LPS array**: Longest proper prefix which is also suffix
- **Manacher's**: Linear time palindrome detection
- **Z-algorithm**: Linear pattern matching

---

# 14. Java Standard Library

**Collections**:

- `Arrays.sort()`, `Collections.sort()`
- `Arrays.binarySearch()`, `Collections.binarySearch()`
- `List.subList(from, to)`, `String.substring(start, end)`

**Data Structures**:

- **Queue**: `LinkedList` or `ArrayDeque`
- **Stack**: `ArrayDeque` (avoid `Stack` class)
- **Heap**: `PriorityQueue` (min heap default)
- **Sorted**: `TreeMap` , `TreeSet`

**Useful Methods**:

- `map.getOrDefault(key, defaultValue)`
- `map.computeIfAbsent(key, mappingFunction)`
- `Collections.reverseOrder()` for max heap
- `Comparator.comparing()` for custom sorting

**Known Tricks**:

- Use `ArrayDeque` instead of `Stack`
- `Character.getNumericValue()` for digit conversion
- `Integer.MAX_VALUE / 2` to avoid overflow in BFS
- `Collections.frequency()` for counting occurrences